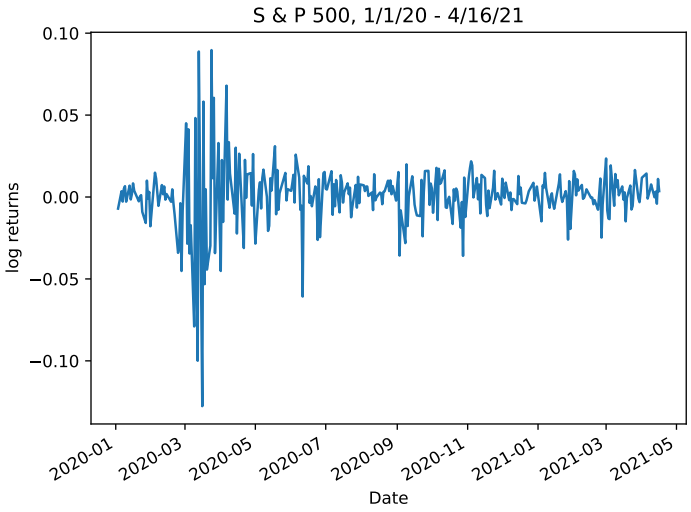
Financial Time Series
46-929
Solutions #4

1. (Forecasting stock prices)
   Download S&P 500 index data for 1/1/2020 to 4/16/2021. Using log-returns check that the time series is weakly stationary. Using data for 1/1/2020 to 4/1/2021 fit an ARIMA model and forecast the index from 4/2/2021 to 4/16/2021.



smt.stattools.adfuller(logret)
(-4.727019298706814, 7.484883471668374e-05, 8, 315, '1%'
%-3.451281394993741 '5%': -2.8707595072926293, '10%': -2.571682118921643, 1224.9259274784883)
The Dickey Fuller statistics gives a p-value of 7.485 e-05 which is far below 1%, hence we clearly reject the unit root and conclude that the series is weakly stationary.

Using the entire data set and the command
smt.tsa.stattools.arma_order_select_ic(logret,ic=('aic','bic')
we find the best fit using either aic or bic to be ARMA(2,2).

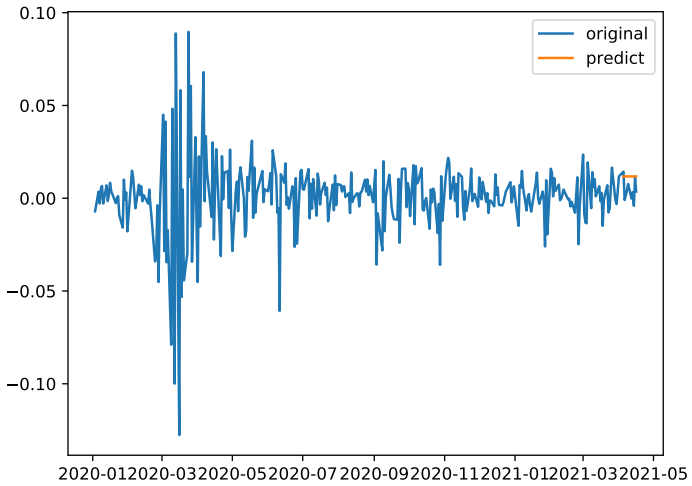
To predict, we divide into a training and testing set and use pmdarima to fit the best model. This turns out to be ARIMA(0,0,2)

train = logret[:'2021-4-1']
test = logret['2020-4-1':]
model=pm.auto_arima(train, seasonal=False, trace = True)
Best model: ARIMA(0,0,2)(0,0,0)[0]

forecast = model.predict(test.shape[0])

The figure below gives a graphic of the S&P 500 and the prediction, done for log-returns. By computing S_and_P[0]_ac(np.exp(np.cumsum(logret)), this will map the log-returns and the prediction to their natural values.

This result is quite disappointing. The problem is that the "best" model is ARIMA(0,0,2), a MA(2) model. Recall that when forecasting with a MA model, once the q parameter is reached (2 in this case), the predictions will revert to the mean. Recognizing this, one should search for a model that gives a reasonable fit and has more predictive power.

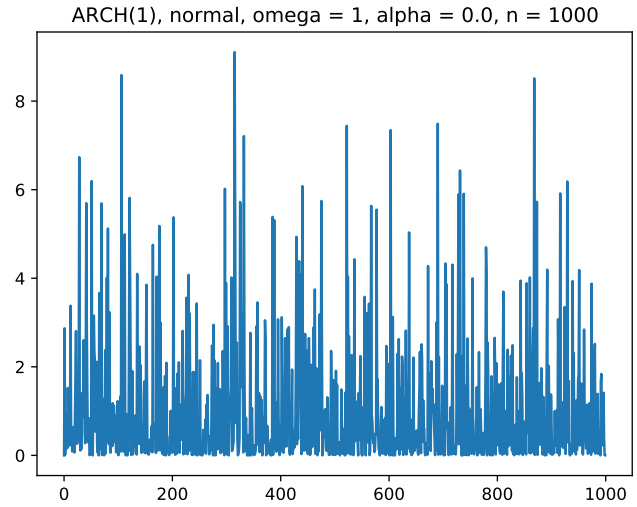


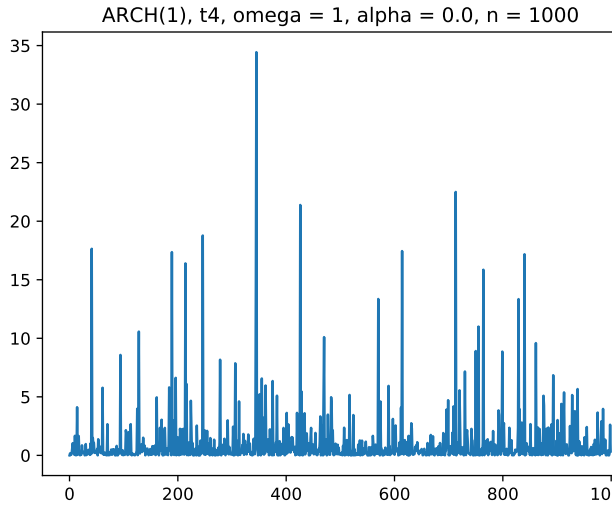
2. (Simulating ARCH(1))
Consider an ARCH(1) model

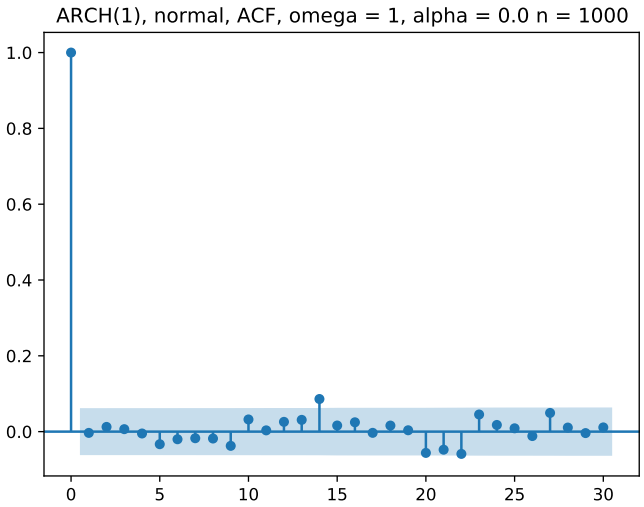
$$a_t = \epsilon_t \sigma_t,$$
$$\sigma_t^2 = 1 + \alpha a_{t-1}^2,$$

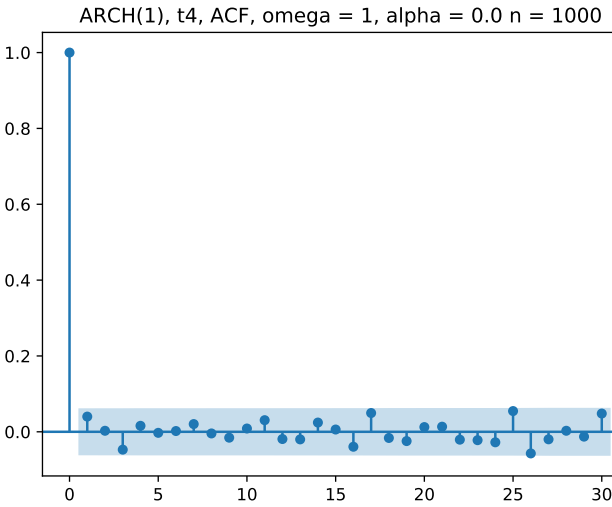
where $\{\epsilon_t\}$ are i.i.d. N(0,1) random variables. Simulate and plot $\{a_t^2,\ 1 \le t \le 1000\}$ assuming $\alpha = .2, .4, .6$. How, if at all, do the plots differ? Do the plots change if the Normal(0,1) distribution is changed to a $t_4/\sqrt{2}$ distribution? (Note, the t_4 distribution has variance 2).

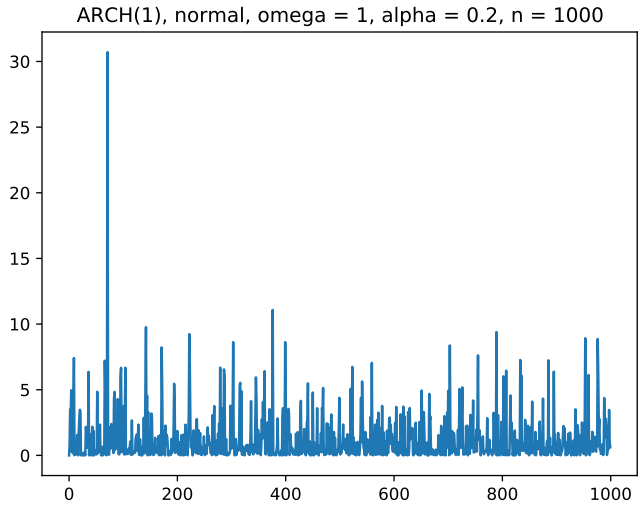
Solution: We present the plots of $\{a_t^2\}$ for both the normal and the $t_4/\sqrt{2}$ innovations and the program below that also provides the acf and pacf plots. We show just the $\{a_t^2\}$ and the acf plots. Plots for $\alpha = 0$ are included, as there are no ARCH effects for this value. The scale is larger for the t-distribution than for the normal and the number of clear volatility clusers increases as alpha increases.

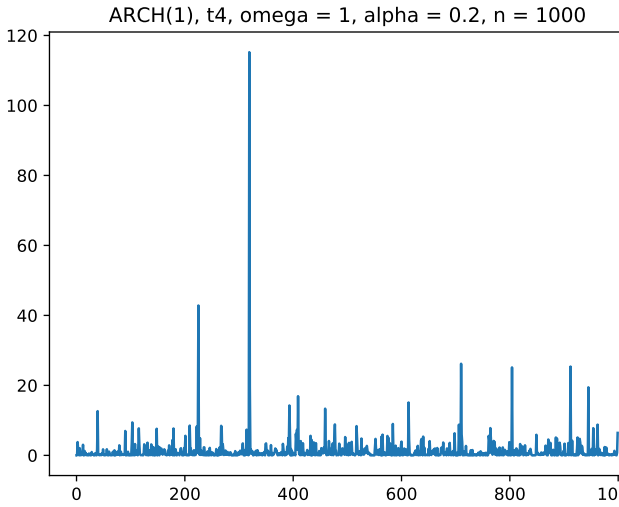

ARCH(1), normal, omega = 1, alpha = 0.0, n = 1000
0
2
4
6
8
0
200
400
600
800
1000

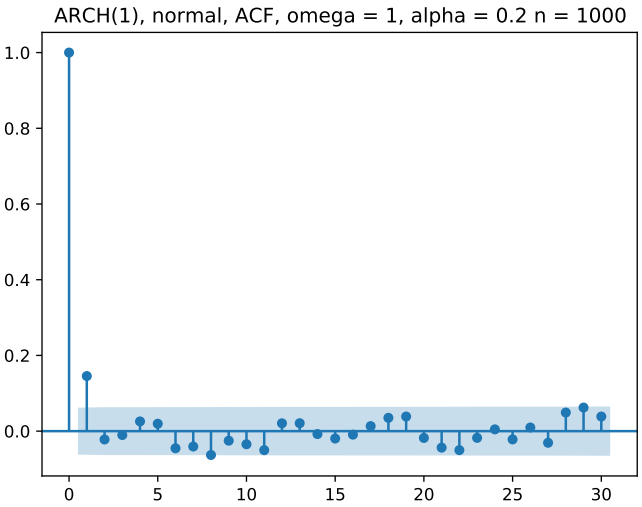

ARCH(1), t4, omega = 1, alpha = 0.0, n = 1000
0
5
10
15
20
25
30
35
0
200
400
600
800

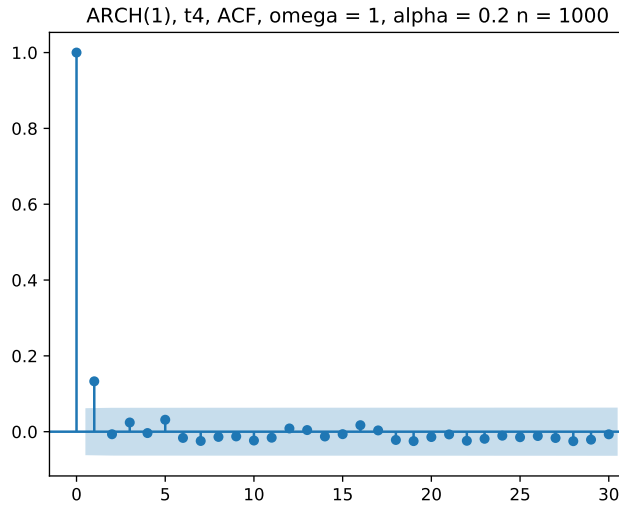

ARCH(1), normal, ACF, omega = 1, alpha = 0.0 n = 1000
0.0
0.2
0.4
0.6
0.8
1.0
0
5
10
15
20
25
30

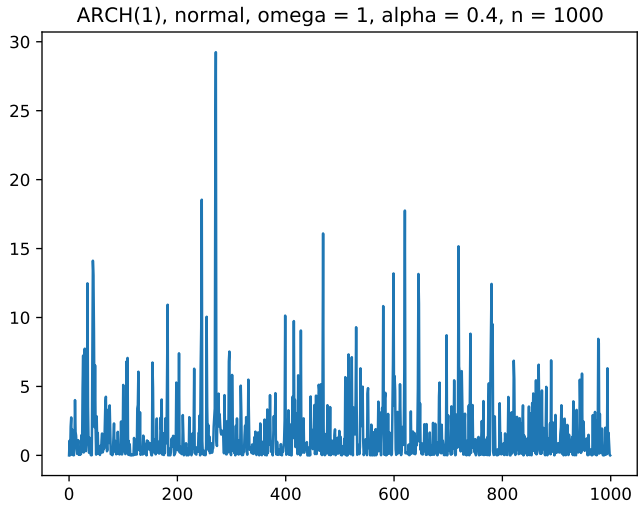

ARCH(1), t4, ACF, omega = 1, alpha = 0.0 n = 1000
0.0
0.2
0.4
0.6
0.8
1.0
0
5
10
15
20
25
30

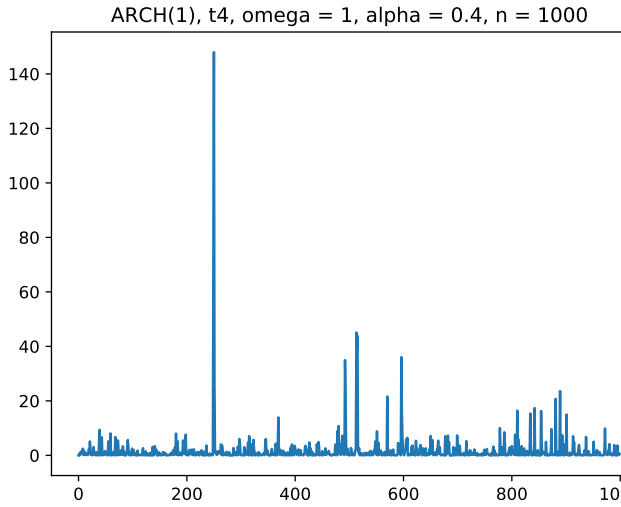

ARCH(1), normal, omega = 1, alpha = 0.2, n = 1000
0
5
10
15
20
25
30
0
200
400
600
800
1000

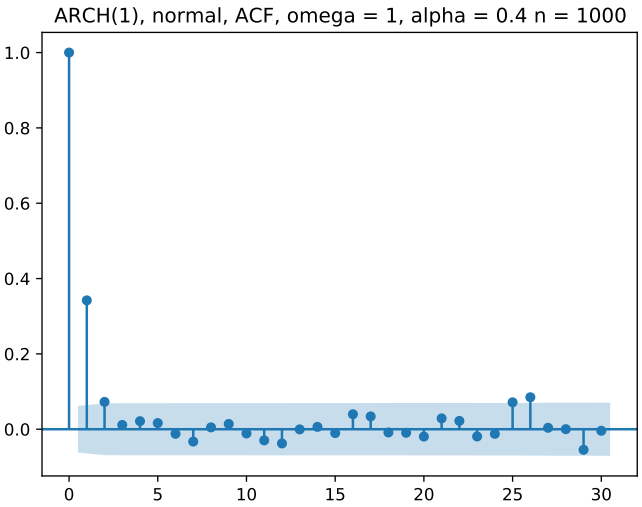

ARCH(1), t4, omega = 1, alpha = 0.2, n = 1000
0
20
40
60
80
100
120
0
200
400
600
800

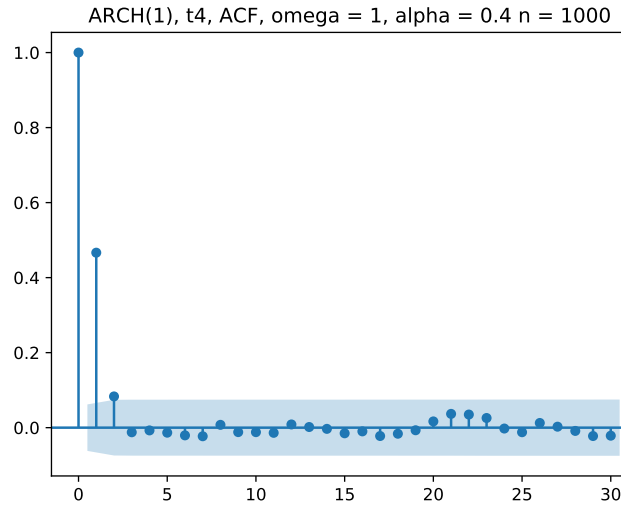

ARCH(1), normal, ACF, omega = 1, alpha = 0.2 n = 1000
0.0
0.2
0.4
0.6
0.8
1.0
0
5
10
15
20
25
30

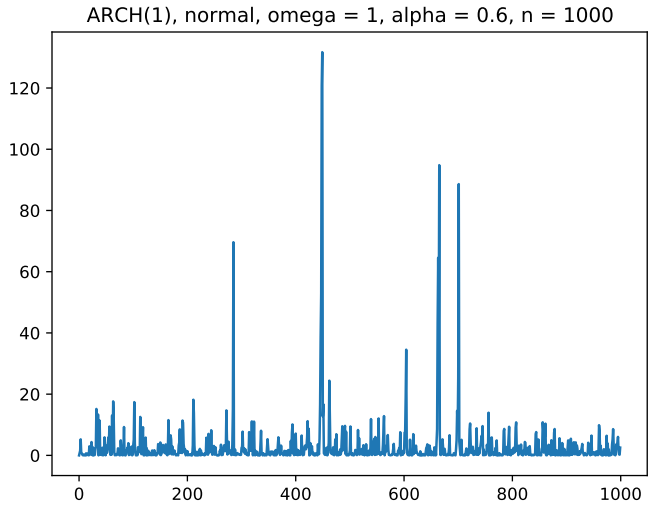

ARCH(1), t4, ACF, omega = 1, alpha = 0.2 n = 1000
0.0
0.2
0.4
0.6
0.8
1.0
0
5
10
15
20
25
30

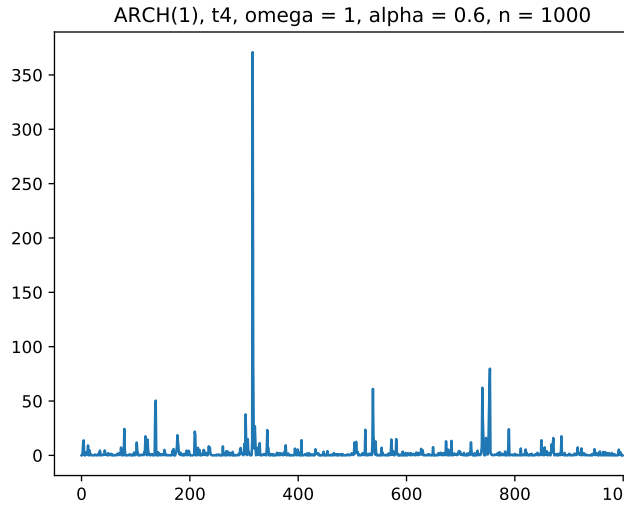

ARCH(1), normal, omega = 1, alpha = 0.4, n = 1000
30
25
20
15
10
5
0
0
200
400
600
800
1000

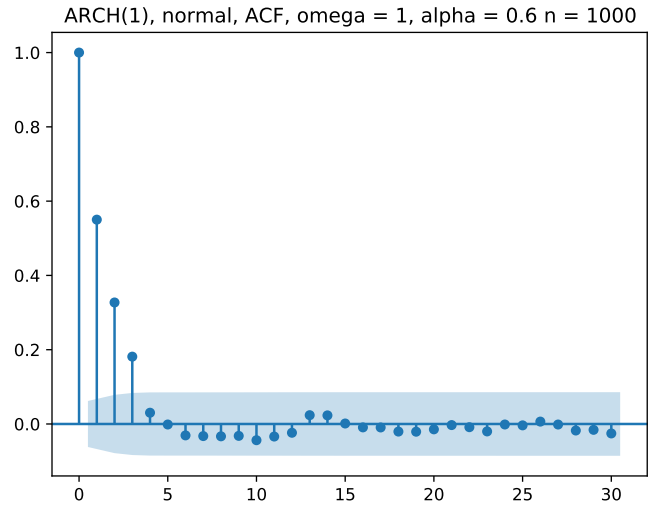

ARCH(1), t4, omega = 1, alpha = 0.4, n = 1000
140
120
100
80
60
40
20
0
0
200
400
600
800

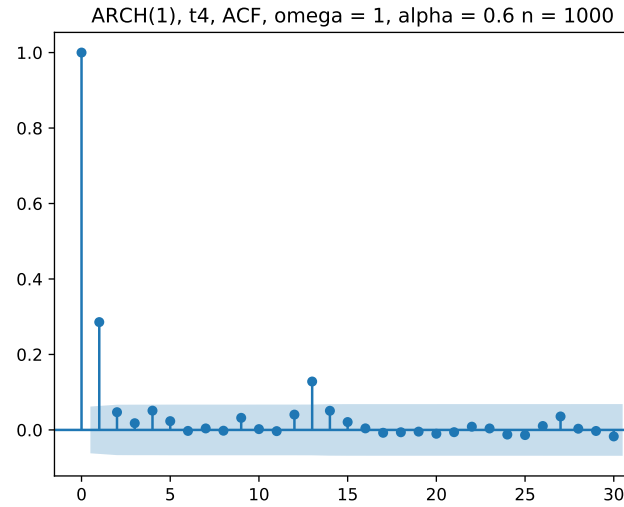

ARCH(1), normal, ACF, omega = 1, alpha = 0.4 n = 1000
1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
25
30


ARCH(1), t4, ACF, omega = 1, alpha = 0.4 n = 1000
1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
25
30










```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels as sm

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf



n = 1000
omega = 1
```

```
alpha = 0.6


epn = np.random.randn(n)
ept = np.random.standard_t(4,n)/np.sqrt(2)
signo = np.zeros(n)
sigt = np.zeros(n)
an = np.zeros(n)
at = np.zeros(n)

for i in range(1,n):
     signo[i] = np.sqrt(omega + alpha*an[i-1]**2)
     an[i] = epn[i]*signo[i]
     sigt[i] = np.sqrt(omega + alpha*at[i-1]**2)
     at[i] = ept[i]*sigt[i]

ansq = an*an
atsq = at*at



plt.plot(ansq)
plt.title('ARCH(1), normal, omega = 1, alpha = 0.6, n = 1000')
plt.show()

plot_acf(ansq)
plt.title('ARCH(1), normal, ACF, omega = 1, alpha = 0.6 n = 1000')
plt.show()

plot_pacf(ansq)
plt.title('ARCH(1), normal, PACF, omega = 1, alpha = 0.6, n = 1000')
plt.show()

plt.plot(atsq)
plt.title('ARCH(1), t4, omega = 1, alpha = 0.6, n = 1000')
plt.show()

plot_acf(atsq)
plt.title('ARCH(1), t4, ACF, omega = 1, alpha = 0.6 n = 1000')
plt.show()

plot_pacf(atsq)
plt.title('ARCH(1), t4, PACF, omega = 1, alpha = 0.6, n = 1000')
plt.show()
```

3. (AR(1) + ARCH(1) Process)
   Suppose that ϵ_t is an iid WN$(0, 1)$ process, that

$$a_t = \epsilon_t \sqrt{1.5 + 0.30 a_{t-1}^2},$$

and that

$$Y_t = 1 + 0.6Y_{t-1} + a_t.$$

a) Find the mean of Y_t.

b) Find the variance of Y_t.

c) Find the autocorrelation function of Y_t.

d) Find the autocorrelation function of a_t.

e) Find the autocorrelation function of a_t^2.

Solutions

a,b) The Y_t process is an AR(1) process with an ARCH(1) noise process with mean 0 and variance σ_a^2. Consequently, we may use previously derived formulas, except we first need to determine σ_a^2. For an ARCH(1) process, $Var(a_t) = \omega/(1.5 - \alpha_1) = 1.5/(1 - .30) = 1.5/.70 = 2.14286$ Consequently

$$E(Y_t) = \phi_0/(1 - \phi_1) = 1/.40 = 2.5.$$

$$Var(Y_t) = \sigma_a^2/(1 - \phi_1^2) = 2.14286/(1 - .6^2) = (1.5/.70)/(1 - .6^2) = 3.3482$$

c) The autocorrelation function also does not depend upon the specific white noise process. $\rho_Y(h) = \phi_1^{|h|}$, so $\rho_Y(h) = .60^{|h|}$

d) The ARCH(1) process $\{a_t\}$ is a white noise process. Hence its ACF, $\rho(h) = 1$ for $h = 0$ and 0 otherwise.

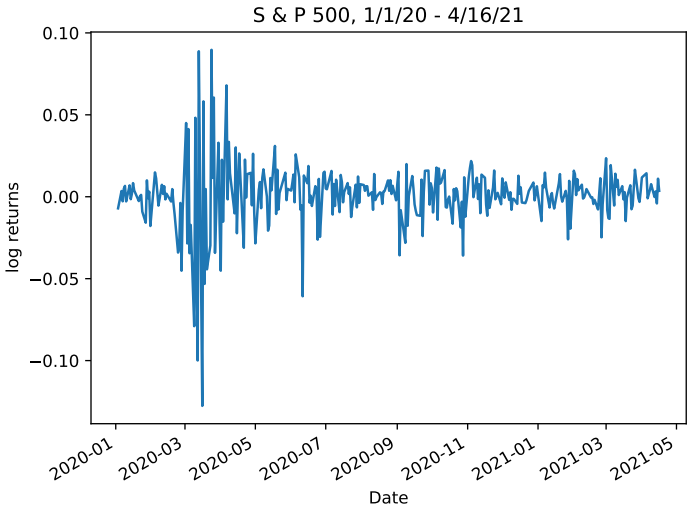
e) $\{a_t^2\}$ is an AR(1) process whose ϕ_1 parameter is $\alpha = 0.30$. Consequently $\rho_{a^2}(h) = (.3)^{|h|}$. process is $\alpha_1^{|h|}$ which is $.30^{|h|}$.

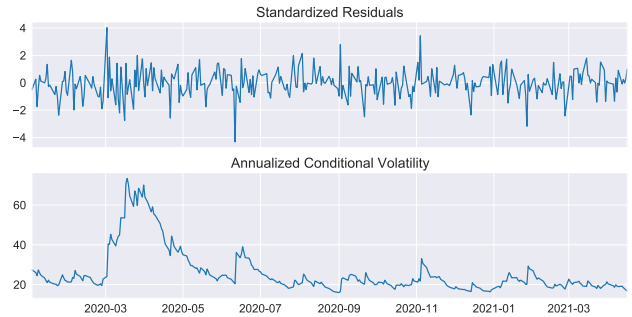
4. (GARCH Modeling)
Using the S&P 500 index data from problem 1, fit a constant mean μ and GARCH volatility model using the ARCH python package. Note the coefficients, their standard error and the p-values for testing if any of them are 0.

Refer to the documents and examples for the ARCH package at
https://arch.readthedocs.io/en/latest/univariate/introduction.html.

This type of problem is covered in the examples.

Solution Following the example given in class (and code at the bottom), the graph of the log returns is given below followed by the standardized residuals and annualized conditional volatility derived from a GARCH(1,1) model with parameter values
$\omega = 0.1381(7.424e - 01)$
$\alpha = 0.1193(4.282e - 02)$
$\beta = 0.a8275(5.793e - 02)$





We check for remaining ARCH effects using the command
smm.stats.diagnostic.het_arch(std_resid)
(8.556480730466218, 0.9531576300464633, 0.4873959829114462, 0.9577436943534197)
The p-values are .953 and .958 for the Engle and the F-test respectively indicating that the ARCH effects have been successfully removed. Finally, we present a probability plot of the standard residuals. Note that there is some evidence of a heavier than normal tail, but not severely so.